TEMA 7 – Introducción al Aprendizaje Computacional

El problema del aprendizaje computacional
- Planteamiento del problema

Conceptos básicos
- Tipos de Aprendizaje
- Fases del Aprendizaje y proceso de inferencia
- Características deseables de un sistema de aprendizaje
- Estimación del error

Algunos sistemas básicos de aprendizaje e inferencia
- Aprendizaje memorístico
- El aprendizaje en la resolución de problemas
- Aprendizaje de reglas de asociación

EL PROBLEMA DEL APRENDIZAJE COMPUTACIONAL

PLANTEAMIENTO DEL PROBLEMA

Con el término *aprendizaje* nos referimos al proceso por el que una entidad acrecienta su conocimiento y/o su habilidad.
Aprendizaje es un aspecto esencial de la inteligencia.
Un programa convencional, sin aprendizaje, ante las mismas entradas siempre generará la misma salida, aunque sea un error.
La *experiencia* es un elemento imprescindible en el aprendizaje.



"*La máquina analítica no tiene intención de originar nada. Puede hacer cualquier cosa siempre y cuando sepamos cómo ordenárselo*" atribuida a Lady Ada Augusta (1815, 1852).

El problema del aprendizaje computacional

Planteamiento del problema

Algunas definiciones de Aprendizaje Computacional.

En el mundo de la psicología:

> *"El aprendizaje constituye un cambio adaptativo observado en el comportamiento del organismo y resulta de la interacción de éste con el medio, siendo inseparable de la maduración fisiológica y de la educación"*

Herbert Simon, en 1983 proporciona la siguiente definición:

> "Learning denotes changes in the system that are adaptive in the sense that they enable the system to do the same task (or tasks drawn from the same population) more effectively the next time"
>
> Machine Learning (Michalski, Carbonell, Mitchell), 1983, Chapter 2

Según esta definición, el aprendizaje se refiere a todo un espectro de actividades, desde refinamiento de la habilidad hasta la adquisición de conocimiento.

El problema del aprendizaje computacional

Planteamiento del problema

El aprendizaje ¿porqué es importante?.

En el mundo de la psicología:

> *"Entender el proceso de aprendizaje es importante. Últimamente se están llegando a modelos de aprendizaje humano comúnmente aceptados por especialistas mediante el uso de modelos de aprendizaje en máquinas"*

En el mundo de las ciencias de la computación:

> Un programa no puede denominarse inteligente si no tiene capacidades de adaptación a cambios. Por tanto, es necesario conseguir programas que tengan la hablidad de aprender.

El problema del aprendizaje computacional

Planteamiento del problema

Algunas razones que justifican el aprendizaje computacional:

1. Algunas tareas solamente pueden describirse bien mediante ejemplos (no conocemos por qué se genera una determinada salida ante una entrada concreta).
2. Es posible que escondidos en una gran cantidad de datos se encuentren relaciones y correlaciones valiosas para nosotros.
3. Algunas máquinas no trabajan tan bien inicialmente como se desearía a largo plazo (parámetros no conocidos en fase de diseño).
4. En ciertos dominios, la cantidad de conocimiento disponible puede sobrepasar la capacidad de codificación por el humano.
5. Los entornos cambian con el tiempo, los programas deberían adaptarse a esos cambios.
6. Constantemente se genera nuevo conocimiento, vocabulario sobre determinadas tareas.

Introduction to Machine Learning, N.J. Nilsson, 1998 (draft of Notes)

CONCEPTOS BÁSICOS

TIPOS DE APRENDIZAJE

Dependiendo de la forma de la experiencia usada para aprender:

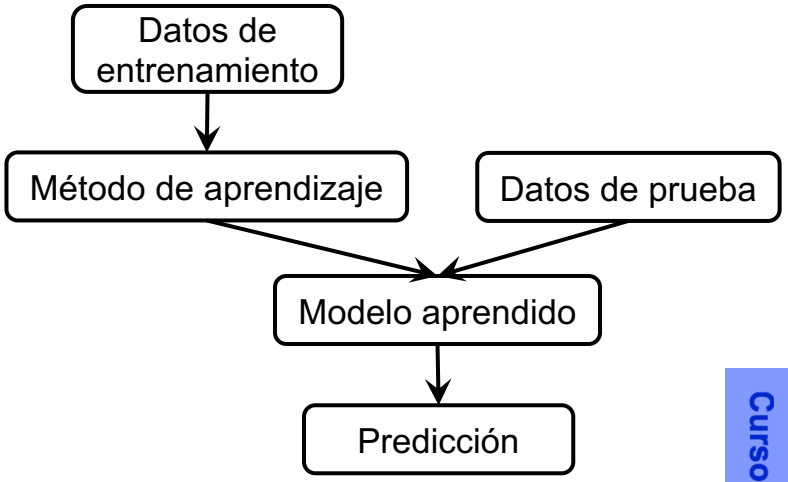
- Aprendizaje supervisado: la experiencia viene en forma de ejemplos etiquetados.
- Aprendizaje no supervisado: la experiencia viene en forma de observaciones del entorno.

FASES DEL APRENDIZAJE Y PROCESO DE INFERENCIA

Fases del aprendizaje: entrenamiento y prueba.

Parámetros externos versus parámetros internos (no aprendidos vs aprendidos).

La base del aprendizaje es la optimización de una función de rendimiento.



Conceptos básicos

Fases del aprendizaje y proceso de inferencia

Un conjunto de ejemplos etiquetados para aprendizaje supervisado.

características o atributos → atributo clase

ejemplos								etiquetas de clase
nada	si	caliente	4	tierra	vivíparos	pulmones		mamífero
nada	si	caliente	0	mar	vivíparos	pulmones		mamífero
pelo	si	caliente	2	mar	ovíparos	pulmones		mamífero
pelo	si	caliente	4	aire	vivíparos	pulmones		mamífero
escamas	no	fría	0	mar	ovíparos	branquias		pez
escamas	no	fría	4	tierra	ovíparos	pulmones		reptil
escamas	no	fría	0	mar	ovíparos	pulmones		reptil
plumas	no	caliente	2	aire	ovíparos	pulmones		pájaro
plumas	no	caliente	2	tierra	ovíparos	pulmones		pájaro
nada	no	fría	4	tierra	ovíparos	pulmones		pájaro

En la fase de entrenamiento se distingue entre supervisión o no supervisión (aprendizaje supervisado o no supervisado).

En la fase de prueba no se hace esa distinción (se habla de clasificación o predicción).

Por último, queda el proceso de inferencia (predicción en la figura anterior).

Conceptos básicos

CARACTERISTICAS DESEABLES DE UN SISTEMA DE APRENDIZAJE

- Precisión: fiabiliad del modelo aprendido, representada normalmente por la proporción de ejemplos aprendidos correctamente (para distringuir por clase, usamos la matriz de confusión).
- Velocidad: rapidez al producir la predicción. A veces es preferible un método con una precisión del 90% a uno del 95% si el primero es 10 veces más rápido (por ejemplo, lecturas de códigos postales o detección de errores en cadenas de producción).
- Comprensión: si es un operador humano el que usa el modelo aprendido, debe ser fácil de entender para evitar errores. En Chernobyl, el sistema recomendó la desconexión pero el operador no creyó que la recomendación estuviese bien fundada.
- Tiempo en aprender: de especial importancia en aprendizaje on-line aplicado a sistemas que cambian rápidamente. También puede implicar la necesidad de un número pequeño de observaciones para obtener un modelo adecuado.

ESTIMACIÓN DEL ERROR

Información básica para estimar el error de un modelo: matriz de confusión.

		Clase predicha: ausente	Clase predicha: presente
Clase real	ausente	80	10
	presente	20	40

Algunos estimadores o medidores del error:

1. Estimador del error de los ejemplos

   Sea N el número de ejemplos y sea E el número de veces que el modelo se equivoca. El estimador del error es E/N.

   Es un buen estimador si los ejemplos de prueba no se usaron para aprender. Cuando el conjunto de ejemplos es suficientemente grande, tomamos un entre un 20 y un 30% de los ejemplos para estimar el error (conjunto de prueba).

2. Estimando el error de resustitución

   Usamos para su estimación el mismo conjunto de datos de entrenamiento

   Aproximación optimista al error real (tiende a subestimarlo).

3. Estimando el error por validación cruzada (cross-validation o CV)
   Se usa para estimar el error de un método, no de un modelo concreto.

   Se aplica en casos donde los datos son escasos.

   1. Dividir el conjunto original de ejemplos S en v partes disjuntas de tamaño semejante $S_1,\ldots,S_v$.
   2. Para $i = 1,\ldots,v$
      a) Construir un modelo usando el método sobre el conjunto de ejemplos $S - S_i$.
      b) Determinar el estimador del error de los ejemplos R_i *usando el conjunto de prueba* S_i.
   3. Calcular el estimador del error por validación cruzada como

$$\sum_{i=1}^{v} \frac{|S_i|}{|S|} R_i$$

   donde $|S|$ y $|S_i|$ indican el número de ejemplos de cada conjunto.

Este método realiza varias estimaciones a partir de diferentes combinaciones de conjuntos de entrenamiento y prueba, y las agrega (más preciso).
Un buen valor de v es 10. Se denomina 10-fold cross-validation.
Un uso particular es el leave-one-out, en donde v=|S|.
Costoso computacionalmente, por lo que se usa con conjuntos pequeños.

Conceptos básicos

Estimación del error

En algunos dominios (por ejemplo, en medicina) algunos errores son más costosos que otros (más costoso errar en un paciente enfermo que en uno sano).

La expresión del costo de cada error se especifica con una matriz de costos similar a la de confusión, c_{ij} es el coste de clasificar un ejemplo de la clase i como de la j:

	ausente	presente
ausente	0	1
presente	5	0

matriz de costos

Si E_{ij} es el número de ejemplos de clase i clasificados como de la clase j, el estimador del costo de la mala clasificación es:

$$c = \sum_{\substack{i=1 \\ i \neq j}}^{N_c} \sum_{j=1}^{N_c} c_{ij} E_{ij}$$

Si usamos la matriz de confusión y la matriz de costes anteriores, tenemos que el costo de la mala clasificación es

C=(20x5 + 10x1)=110

	ausente	presente
ausente	80	10
presente	20	40

matriz de confusión

Estimación del error

Cuando el atributo clase es numérico, y no categórico como hasta ahora, un buen estimador del error es el error cuadrático medio (MSE o Mean Squared Error):

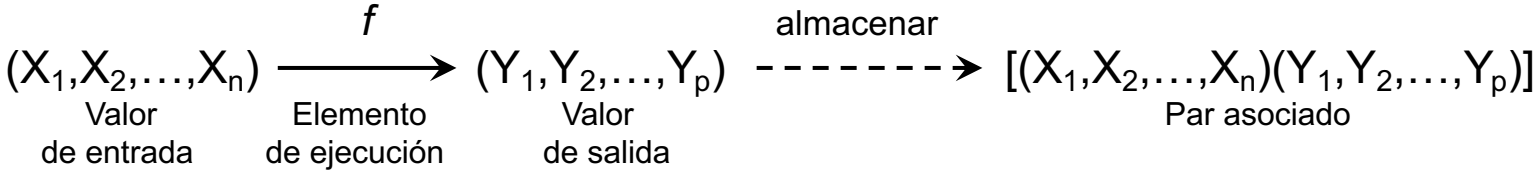
$$\text{MSE}=\frac{\sum_{i=1}^{N}(y_i^*-y_i)^2}{N}$$

donde y_i es el valor inferido por el modelo e y^*_i el valor real.

ALGUNOS SISTEMAS BÁSICOS DE APRENDIZAJE E INFERENCIA

APRENDIZAJE MEMORÍSTICO

La idea básica es almacenar todo el nuevo conocimiento para utilizarlo cuando sea preciso. Todo sistema de aprendizaje debe recordar lo aprendido para poder utilizarlo en el futuro, por tanto esta parte puede estar integrada en un sistema aprendedor más complejo.



Toma problemas ya resueltos por el sistema (elemento de ejecución) y memoriza el problema + su solución

Consideremos el diseño de un programa que determina el coste de reparaciones de coches siniestrados para una compañía de seguros:

- Entrada: descripción del coche y sus daños.
- Salida: coste estimado de reparación.
- Se usa la memoria si hay un caso similar, o reglas de la compañía en otro caso.

Algunos sistemas básicos de aprendizaje e inferencias

Aprendizaje memorístico

En el aprendizaje memorístico podemos destacar algunos elementos:

1. Organización de la memoria.
   Se ha de utilizar memorización siempre que sea más conveniente almacenar que recalcular (indexación, ordenación y hashing).
2. Estabilidad del entorno.
   Cuando el entorno cambia rápidamente, lo almacenado puede quedar rápidamente desfasado, por tanto no es conveniente en este caso.
3. Almacenamiento frente a recálculo.
   Aprendizaje por memorización debe ser eficiente.
   Ningún ordenador almacena una tabla de multiplicar.
   Dos enfoques para conseguir un almacenamiento eficiente.
   Decidir si almacenar o no cada vez que llega nueva información (análisis costo-beneficio).
   Almacenar todo y olvidar después lo que se usa menos frecuentemente (olvido selectivo).

Algunos sistemas básicos de aprendizaje e inferencias

APRENDIZAJE EN LA RESOLUCIÓN DE PROBLEMAS

Un programa para resolver problemas puede aprender a partir de la generalización de sus propias experiencias:

- Tras resolver un problema de interés, se recuerda la estructura del problema, los métodos que se utilizaron para resolverlo y su solución.
- En la siguiente ocasión podemos utilizar esta experiencia directamente, y además, podemos generalizarla para solucionar otros problemas similares.

Un ejemplo sencillo, utilizados por primera vez en STRIPS: uso de **macro-operadores.**

Tras cada episodio de planificación, un componente de STRIPS toma el plan calculado, o una parte, y lo transforma en un macro-operador.

Macro-operador: plan formado por una secuencia de acciones, encapsulado en un operador más abstracto que almacena para sucesivos usos.

Se trata como un operador normal.

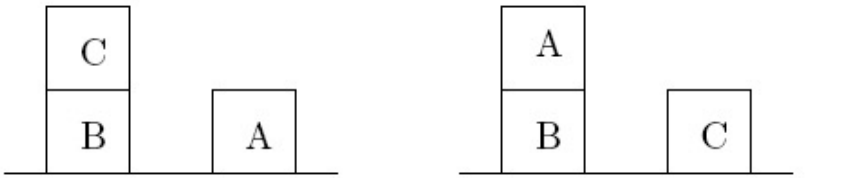
Al general el macro-operador, se construyen:

- Precondiciones: las condiciones iniciales del problema resuelto.
- Postcondiciones: el objetivo alcanzado.

Algunos sistemas básicos de aprendizaje e inferencias

Aprendizaje en la resolución de problemas

Supongamos un problema del tema anterior.



Strips obtiene el plan: DESAPILAR(C,B), DEJAR(C), COGER(A), APILAR(A,B)
Se genera un macro-operador con:

Precondiciones: SOBRE(C,B), SOBRE(A,Mesa)

Postcondiciones: SOBRE(C,Mesa), SOBRE(A,B)

Plan: DESAPILAR(C,B), DEJAR(C), COGER(A), APILAR(A,B)

Raramente se presenta el mismo problema dos veces: se ha de generalizar transformando el macro-operador en uno más genérico que pueda ser equiparado con una familia de problemas → cambiar constantes por variables:

Precondiciones: SOBRE(x1,x2), SOBRE(x3,Mesa)
Postcondiciones: SOBRE (x1,Mesa), SOBRE(x3,x2)
Plan: DESAPILAR(x1,x2), DEJAR(x1), COGER(x3), APILAR(x3,x2)

Algunos sistemas básicos de aprendizaje e inferencias

Aprendizaje en la resolución de problemas

A menudo la generalización no es tan directa, y algunas constantes deben retener su valor. Esto plantea otros problemas.

Supongamos que, en nuestro dominio, existe el operador APILAR-EN-B(X):

Precondiciones: LIBRE(x), LIBRE(B)

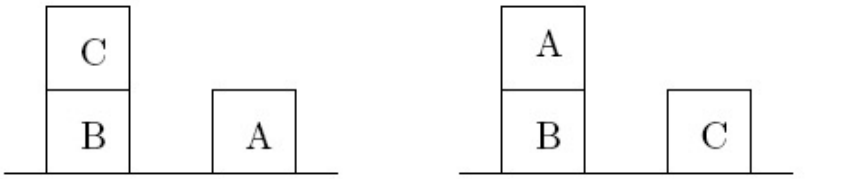
Postcondiciones: SOBRE(x,B)

Supongamos el mismo ejemplo anterior. STRIPS generaría el macro-operador:

Precondiciones: SOBRE(x1,x2), SOBRE(x3, Mesa)

Postcondiciones: SOBRE(x1,Mesa), SOBRE(x3,x2)

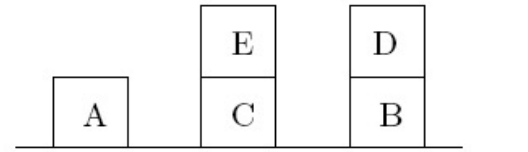
Plan: DESAPILAR(x1,x2), DEJAR(x1), APILAR-EN-B(x3)



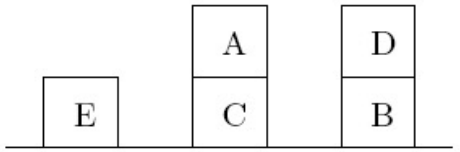
Algunos sistemas básicos de aprendizaje e inferencias

Aprendizaje en la resolución de problemas

Supongamos ahora este problema.



Inicio: SOBRE(E,C)∧SOBRE(D,B)



Objetivo: SOBRE(A,C)

STRIPS intenta usar el marco-operador:

Precondiciones: SOBRE(x1,x2), SOBRE(x3, Mesa)
Postcondiciones: SOBRE(x1,Mesa), SOBRE(x3,x2)
Plan: DESAPILAR(x1,x2), DEJAR(x1), APILAR-EN-B(x3)

STRIPS unifica las variables x1=E, x2=C, x3=A. Se satisfacen las precondiciones y se construye un plan:

{DESAPILAR(E,C), DEJAR(E), APILAR-EN-B(A)} → No funciona!!

Este marco-operador sólo es útil para apilar bloques sobre B: no nos vale.
La dificultad surge cuando se intenta el último paso:

- Aunque se despejó C, que es donde se quería poner A, no se despejó B, que es donde el macro-operador va a tratar de ponerlo.
- Como B no está despejado, APILAR-EN-B no se puede ejecutar. Si B estuviese despejado, el macro-operador se podría ejecutar, pero no alcanzaría el estado objetivo.

El problema es que la postcondición del macro-operador está sobregeneralizada.

Algunos sistemas básicos de aprendizaje e inferencias

Aprendizaje en la resolución de problemas

El proceso de generalización que aplica STRIPS es más complejo:

- Primero, sustituye todas las constantes por variables.
- Después, reevalúa todas las precondiciones de cada operador del plan parametrizado.

En el ejemplo previo, la única de forma de asegurarse de que B está despejado para aplicar el tercer operador (APILAR-EN-B(x3)) es comprobar que el bloque x2 despejado por el primer operador (DESAPILAR(x1,x2)) es realmente el bloque B.

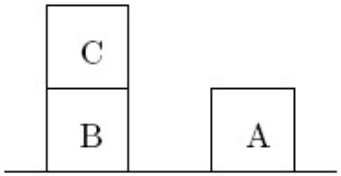
Los macro-operadores, bien generalizados, son muy útiles: un macro-operador puede dar lugar a un pequeño cambio global en el mundo, aunque los operadores individuales que lo forman produzcan muchos cambios locales no buscados.

Lo vemos con una secuencia de ejemplos.

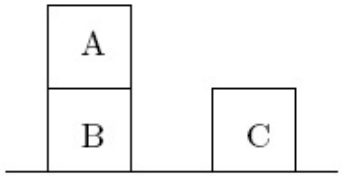
Algunos sistemas básicos de aprendizaje e inferencias

Aprendizaje en la resolución de problemas

Ejemplo 1



Inicio: SOBRE(C,B), SOBREMESA(A), SOBREMESA(B), LIBRE(A), LIBRE(C)



Objetivo: SOBRE(A,B), SOBREMESA(B), SOBREMESA(C), LIBRE(A), LIBRE(C)

$$SOBRE(C,B) \wedge SOBREMESA(A) \wedge LIBRE(A) \wedge LIBRE(C) \wedge SOBREMESA(B)$$

$$\downarrow \quad \boxed{DESAPILAR(C,B),\ DEJAR(C),\ COGER(A),\ APILAR(A,B)}$$

$$SOBRE(A,B) \wedge SOBREMESA(B) \wedge LIBRE(A) \wedge LIBRE(C) \wedge SOBREMESA(C)$$

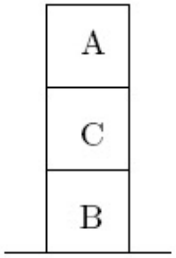
$$\begin{cases} NUEVO(A,B) : DESAPILAR(C,B),\ DEJAR(C),\ COGER(A),\ APILAR(A,B) \\ P: SOBRE(C,B) \wedge SOBREMESA(A) \wedge LIBRE(A) \wedge LIBRE(C) \wedge SOBREMESA(B) \\ S: SOBREMESA(A) \wedge SOBRE(C,B) \\ A: SOBREMESA(C) \wedge SOBRE(A,B) \end{cases}$$

Se generaliza a un macro-operador con variables X, Y y Z.

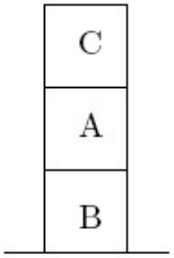
Algunos sistemas básicos de aprendizaje e inferencias

Aprendizaje en la resolución de problemas

Ejemplo 2



Inicio: SOBRE(C,B), SOBRE(A,C), SOBREMESA(B), LIBRE(A)



Objetivo: SOBRE(A,B), SOBRE(C,A), SOBREMESA(B), LIBRE(C)

$$SOBRE(C,B) \wedge SOBRE(A,C) \wedge SOBREMESA(B) \wedge LIBRE(A)$$

$$\downarrow \quad DESAPILAR(A,C),\ DEJAR(A),\ NUEVO(A,B),\ COGER(C),\ APILAR(C,A)$$

$$SOBRE(C,A) \wedge SOBRE(A,B) \wedge SOBREMESA(B) \wedge LIBRE(C)$$

$$NUEVO(X,Y) \quad : \quad DESAPILAR(Z,Y),\ DEJAR(Z),\ COGER(X),\ APILAR(X,Y)$$

$$P:\ SOBRE(Z,Y) \wedge SOBREMESA(X) \wedge LIBRE(X) \wedge LIBRE(Z) \wedge SOBREMESA(Y)$$

$$S:\ SOBREMESA(X) \wedge SOBRE(Z,Y)$$

$$A:\ SOBREMESA(Z) \wedge SOBRE(X,Y)$$

Algunos sistemas básicos de aprendizaje e inferencias

APRENDIZAJE DE REGLAS DE ASOCIACIÓN

Una regla de asociación relaciona los distintos elementos que pueden aparecer en una base de datos relacional o transaccional. Una regla de asociación, a diferencia de las reglas de un SBR, no expresa causalidad (causa-efecto).

Un ejemplo de regla de asociación puede ser la siguiente:

SI leche, galletas ENTONCES pañales

Las reglas de asociación tienen aplicaciones en:

- Soporte para la toma de decisiones.
- Diagnóstico y predicción.
- Análisis de información de ventas.
  - Diseño de catálogos.
  - Distribución de mercancías en tiendas.
  - Segmentación de clientes en base a patrones de compra.

Algunos sistemas básicos de aprendizaje e inferencias

Aprendizaje de reglas de asociación

Las reglas de asociación se obtienen a partir de observaciones:

- Aquellas cuyo antecedente y consecuente son ciertos en muchas de las tuplas tendrán una cobertura (soporte) alta.
- Aquellas cuando se cumple el antecedente, también lo hace el consecuente en muchas de las tuplas tendrán una confianza alta (precisión).

Nuestro objetivo es encontrar reglas interesantes, con valores buenos de cobertura y confianza.

Estrategia: usando covering en el LHS (left-hand side), considerar cada combinación (atributo, valor) en el RHS (right-hand side), quedándonos con aquellas que son interesantes.

Algunos sistemas básicos de aprendizaje e inferencias

Aprendizaje de reglas de asociación

Notación necesaria:

- $I=\{i_1,i_2,\ldots,i_n\}$ conjunto de n atributos binarios, llamados ítems.
- $D=\{t_1,t_2,\ldots,t_m\}$ conjunto de m ejemplares (transacciones) de un problema.
- t_i, también denominado itemset, es un conjunto de items de I.

Una regla de asociación es una expresión de la forma **Si X entonces Y** (o **$X \Rightarrow Y$**) donde X e Y son itemsets y $X \subseteq I$, $Y \subseteq I$, $X \cap Y = \varnothing$.

- Confianza (precisión): La confianza de una regla $X \Rightarrow Y$ se interpreta como la probabilidad de que una transacción que contiene los ítems de X, también contenga los ítems de Y. $c(X \Rightarrow Y)=P(Y/X)=|X \cup Y|/|X|$
- Soporte (cobertura): El soporte de una regla $X \Rightarrow Y$ (o del itemset $X \cup Y$) se interpreta como la probabilidad de que una transacción contenga a todos los ítems de X y de Y. $s(X \Rightarrow Y)=P(X \cup Y)=|X \cup Y|/|D|$

Nota: Generalmente, tanto el soporte como la confianza se expresan en %.

Estrategia: estamos interesados en reglas con mucho soporte, por lo que buscamos pares (atributo, valor) que cubran gran cantidad de transacciones (itemsets). Una vez tenemos los itemsets, los transformamos en reglas.

Algunos sistemas básicos de aprendizaje e inferencias

Aprendizaje de reglas de asociación

Transacción	Elementos comprados
1	A, B, C
2	A, C
3	A, D
4	B, E, F

Soporte mínimo 50% y la confianza mínima 50%.

Lo cumplen

$A \Rightarrow C$ (c=2/3, 66.6%; s=2/4, 50%);
$C \Rightarrow A$ (c=2/2, 100%;s=2/4, 50%), …

No lo cumplen

$A \Rightarrow B$ (c=1/3, 33.3%;s=1/4, 25%);
$A,B \Rightarrow C$ (c=1/1, 100%; s=1/4, 25%),
$A,C \Rightarrow B$ (c=1/2, 50%; s=1/4,25%); …

Algunos sistemas básicos de aprendizaje e inferencias

Aprendizaje de reglas de asociación

Dos fases para obtener las reglas.

Fase 1

Generar todas las combinaciones de items, entendiendo éstas como itemsets, con un soporte por encima del mínimo (e.d. frequent itemsets).

Fase 2

Dado un itemset frecuente $Y=\{i_1,i_2,\ldots,i_k,\ k \geq 2\}$, generar todas las reglas que contengan todos los items de ese itemset, generando todos los $X \Rightarrow (Y-X)$ que cumplan que su confianza ($c(X \Rightarrow (Y-X))=s(X \cup Y)/s(X)$) es mayor que la mínima requerida.

Algoritmo A PRIORI

R. Agrawal, T. Imielinski, A. Swami. Mining association rules between sets of items in large databases. Proc. ACM SIGMOD Int. Conf. on Management of Data, 1993.

Definimos los itemsets frecuentes como aquellos itemsets con soporte mínimo.

Propiedades equivalentes:

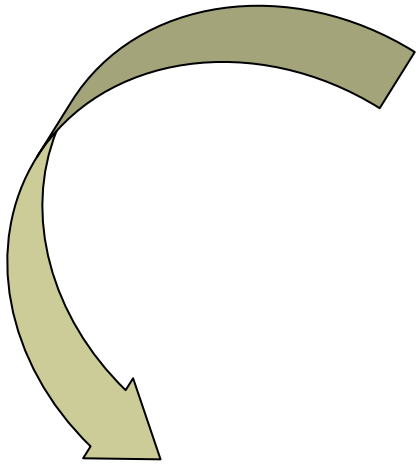
- Un subconjunto de un itemset frecuente es también un itemset frecuente.
- Si un itemset no satisface el soporte mínimo, ningún superconjunto lo satisface.

Iterativamente, se encuentran itemsets frecuentes con cardinalidad 1 hasta k.

Algunos sistemas básicos de aprendizaje e inferencias

Aprendizaje de reglas de asociación

FASE 1



función APRIORI-CANDIDATOS(L_k)

Fase de Formación:

insertar en C_{k+1} todas las combinaciones de los itemsets desde L_k para formar itemsets de tamaño k+1

Fase de Poda:

para todo itemsets $c_i \in C_{k+1}$ **hacer**

para todos k-subconjuntos s de c_i **hacer**

si ($s \notin L_k$) **entonces** borrar c_i de C_{k+1}

función APRIORI

C_k: itemsets candidatos de tamaño *k*

L_k: itemsets frecuentes de tamaño *k*

L_1={itemsets frecuentes de tamaño 1}

para (k=1; L_k <> ∅ ; k++) **hacer**

begin

C_{k+1}=apriori-candidatos(L_k);

para todos transacción t∈D **hacer**

begin

C_t=subset(C_{k+1},t);

para todos candidatos $c_i \in C_t$ **hacer** c_i.count++;

end

L_{k+1}={$c_i \in C_{k+1}$ / c_i.count ≥ min_soporte}

end

devolver $\cup_k L_k$

Algunos sistemas básicos de aprendizaje e inferencias

Aprendizaje de reglas de asociación

Ejemplo: Sea un conjunto de datos como el de la tabla. Fijamos umbrales:

min_soporte = 30% min_confianza = 75%

Para encontrar los itemset frecuentes (Fase 1), aplicamos algoritmo APRIORI.

Conjunto de transacciones D

ID	Itemsets
1	p_1, p_2, p_5
2	p_2, p_4
3	p_2, p_3
4	p_1, p_2, p_3
5	p_1, p_3
6	p_2, p_3, p_5
7	p_1, p_3
8	p_1, p_2, p_3, p_5
9	p_1, p_2, p_3

- Partimos de:
  - $I=\{p_1, p_2, p_3, p_4, p_5\}$
  - $D=\{ \{p_1,p_2,p_5\}, \{p_2,p_4\}, \{p_2,p_3\}, \{p_1,p_2,p_3\}, \{p_1,p_3\}, \{p_2,p_3,p_5\}, \{p_1,p_3\}, \{p_1,p_2,p_3,p_5\}, \{p_1,p_2,p_3\} \}$
- Construimos L_1: Itemset frecuentes de cardinalidad 1

Itemset	Sop. × |D|
p_1	6 ●
p_2	7 ●
p_3	7 ●
~~p_4~~	~~1~~ ●
p_5	3 ●

Se comprueba el umbral de soporte.

Obsérvese que min_soporte x |D| es 2.7 donde |D| es el cardinal o número de transacciones del conjunto D.

Algunos sistemas básicos de aprendizaje e inferencias

Aprendizaje de reglas de asociación

A partir de L_1, construimos C_2: algoritmo APRIORI-CANDIDATOS (L_1).

- Candidatos de cardinalidad 2: Todas las combinaciones de 2 elementos de L_1.
- Eliminamos combinaciones cuyos subconjuntos no estén todos en L_1.

ID	Itemsets
1	p_1, p_2, p_5
2	p_2, p_4
3	p_2, p_3
4	p_1, p_2, p_3
5	p_1, p_3
6	p_2, p_3, p_5
7	p_1, p_3
8	p_1, p_2, p_3, p_5
9	p_1, p_2, p_3

L_1

Itemset	Sop. × |D|
p_1	6
p_2	7
p_3	7
p_5	3

C_2

Itemset
p_1, p_2
p_1, p_3
p_1, p_5
p_2, p_3
p_2, p_5
p_3, p_5

- Para cada transacción t en D, extraemos los candidatos de C_2 que sean subconjuntos de t, e incrementamos su cuenta en una unidad, obteniendo L_2.

L_2

Itemset	Sop. × |D|
p_1, p_2	4 ●
p_1, p_3	5 ●
~~p_1, p_5~~	~~2~~ ●
p_2, p_3	5 ●
p_2, p_5	3 ●
~~p_3, p_5~~	~~2~~ ●

Se comprueba el umbral de soporte

Algunos sistemas básicos de aprendizaje e inferencias

Aprendizaje de reglas de asociación

A partir de L_2, construimos C_3: algoritmo APRIORI-CANDIDATOS (L_2).

- Candidatos de cardinalidad 3: Todas las combinaciones de 3 elementos de L_2.
- Eliminamos combinaciones cuyos subconjuntos no estén todos en L_2.

ID	Itemsets
1	p_1, p_2, p_5
2	p_2, p_4
3	p_2, p_3
4	p_1, p_2, p_3
5	p_1, p_3
6	p_2, p_3, p_5
7	p_1, p_3
8	p_1, p_2, p_3, p_5
9	p_1, p_2, p_3

L_2

Itemset	Sop. × |D|
p_1, p_2	4
p_1, p_3	5
p_2, p_3	5
p_2, p_5	3

- Para cada transacción t en D, extraemos los candidatos de C_3 que sean subconjuntos de t, e incrementamos su cuenta en una unidad, obteniendo L_3.

C_3

Itemset
p_1, p_2, p_3
~~p_1, p_2, p_5~~
~~p_2, p_3, p_5~~
~~p_1, p_3, p_5~~

No verifican propiedad Apriori

L_3

Itemset	Sop. × |D|
p_1, p_2, p_3	3

Se comprueba el umbral de soporte

No hay más superconjuntos: FIN

Aprendizaje de reglas de asociación

FASE 2 .- Generación de reglas:

- Dado un itemset frecuente $Y=\{i_1,i_2,\ldots,i_k;\ k \geq 2\}$, generar todas las reglas que contengan todos los items de ese itemset:
  - Se toman todos los subconjuntos no vacíos X de Y.
  - Se generan las reglas $X \Rightarrow (Y-X)$ que cumplan que su confianza es mayor que la mínima requerida:
    - $c(X \Rightarrow (Y-X)) = s(X \cup (Y-X)) / s(X) = s(Y) / s(X)$, donde $s(Z) = |Z| / |D|$

función GENERAR-REGLAS-APRIORI

para cada itemset frecuente, *f*, generar todos los subconjuntos no vacíos de *f* (con $k \geq 2$)
para todos subconjuntos no vacios s de *f* **hacer**
si soporte(*f*)/soporte(s) ≥ min_confianza **entonces** regla de salida $s \Rightarrow$ (f-s)

Algunos sistemas básicos de aprendizaje e inferencias

Aprendizaje de reglas de asociación

función GENERAR-REGLAS-APRIORI
  para cada itemset frecuente, *f*, generar todos los subconjuntos no vacíos de *f* (con k≥2)
  para todos subconjuntos no vacios s de *f* **hacer**
    si soporte(*f*)/soporte(s) ≥ min_confianza **entonces** regla de salida s ⇒ (f-s)

Continuamos el ejemplo.

ID	Itemsets
1	p_1, p_2, p_5
2	p_2, p_4
3	p_2, p_3
4	p_1, p_2, p_3
5	p_1, p_3
6	p_2, p_3, p_5
7	p_1, p_3
8	p_1, p_2, p_3, p_5
9	p_1, p_2, p_3

Itemset frecuentes: $p_1(6)$, $p_2(7)$, $p_3(7)$, $p_5(3)$, $p_1p_2(4)$, $p_1p_3(5)$, $p_2p_3(5)$, $p_2p_5(3)$, $p_1p_2p_3(3)$

Reglas candidatas:

Por ejemplo, el primer itemset con k ≥ 2 es: $p_1p_2(4)$:

- Formamos sus subconjuntos: p_1 y p_2
- Formamos las reglas candidatas:
  - $\{p_1\} \Rightarrow \{p_1, p_2\} - \{p_1\}$, es decir, $p_1 \Rightarrow p_2$
  - $\{p_2\} \Rightarrow \{p_1, p_2\} - \{p_2\}$, es decir, $p_2 \Rightarrow p_1$
- Calculamos la confianza:
  - $p_1 \Rightarrow p_2$: $c(p_1 \Rightarrow p_2) = s(p_1,p_2) / s(p_1) = 4/6 = 66\%$
  - $p_2 \Rightarrow p_1$: $c(p_2 \Rightarrow p_1) = s(p_1,p_2) / s(p_2) = 4/7 = 57\%$
- Ninguna supera el umbral de confianza 75%: rechazadas

Algunos sistemas básicos de aprendizaje e inferencias

Aprendizaje de reglas de asociación

función GENERAR-REGLAS-APRIORI

para cada itemset frecuente, *f*, generar todos los subconjuntos no vacíos de *f* (con k≥2)
para todos subconjuntos no vacios s de *f* **hacer**
si soporte(*f*)/soporte(s) ≥ min_confianza **entonces** regla de salida s ⇒ (f-s)

ID	Itemsets
1	p_1, p_2, p_5
2	p_2, p_4
3	p_2, p_3
4	p_1, p_2, p_3
5	p_1, p_3
6	p_2, p_3, p_5
7	p_1, p_3
8	p_1, p_2, p_3, p_5
9	p_1, p_2, p_3

Itemset frecuentes: $p_1(6)$, $p_2(7)$, $p_3(7)$, $p_5(3)$, $p_1p_2(4)$, $p_1p_3(5)$, $p_2p_3(5)$, $p_2p_5(3)$, $p_1p_2p_3(3)$

Reglas candidatas:

k=2:
$p_1 \Rightarrow p_2$ (4/6 = 66%)
$p_2 \Rightarrow p_1$ (4/7 = 57%)
$p_1 \Rightarrow p_3$ (5/6 = 83%)
$p_3 \Rightarrow p_1$ (5/7 = 71%)
$p_2 \Rightarrow p_3$ (5/7 = 71%)
$p_3 \Rightarrow p_2$ (5/7 = 71%)
$p_2 \Rightarrow p_5$ (3/7 = 42%)
$p_5 \Rightarrow p_2$ (3/3 = 100%)

k=3:
$p_1p_2 \Rightarrow p_3$ (3/4 = 75%)
$p_1p_3 \Rightarrow p_2$ (3/5 = 60%)
$p_2p_3 \Rightarrow p_1$ (3/5 = 60%)
$p_1 \Rightarrow p_2p_3$ (3/6 = 50%)
$p_2 \Rightarrow p_1p_3$ (3/7 = 42%)
$p_3 \Rightarrow p_1p_2$ (3/7 = 42%)

Se aceptan las que superan min_confianza = 75 %:

$p_1 \Rightarrow p_3$ (5/6 = 83%)
$p_1p_2 \Rightarrow p_3$ (3/4 = 75%)
$p_5 \Rightarrow p_2$ (3/3 = 100%)